

 Scikit-learn +1

```

# A GridSearchCV(Fit, ParameterTrials, cv=5, n_jobs=
# n_threads) Grid_Search_Fit(F,K):
5 folds for each of 20 candidates, totalling 100 fits
iter=0; max_depth=2, n_estimators=100 .....
n1(n_jobs=1): Using Backward SequentialBackward with 1
iter=0; max_depth=2, n_estimators=100, score=0.
iter=0; max_depth=2, n_estimators=100 .....
iter=0; max_depth=2, n_estimators=100 .....
iter=0; max_depth=2, n_estimators=100 .....
n1(n_jobs=1): Done 1 out of 1 | elapsed: 0.2s
n1(n_jobs=1): Done 2 out of 2 | elapsed: 0.4s
iter=0; max_depth=2, n_estimators=100, score=1
iter=0; max_depth=2, n_estimators=100 .....
iter=0; max_depth=2, n_estimators=100, score=0.
iter=0; max_depth=2, n_estimators=100 .....
n1(n_jobs=1): Done 3 out of 3 | elapsed: 0.6s

```

- **Grid Search:** You give it a list of settings (hyperparameters) and values to try. It tests every single combination.
- **Cross-Validation (CV):** It splits your training data into smaller parts called folds. It trains the model on some folds and tests it on the remaining fold. This repeats so every fold gets a turn as the test set.
- **Selection:** It scores each combination and picks the set of settings with the highest average score.

- **estimator** : The machine learning model you want to use (like a Random Forest or Support Vector Machine).
- **param_grid** : A dictionary where keys are setting names and values are lists of settings to test.
- **cv** : The number of splits (folds) used for cross-validation.
- **scoring** : The metric used to judge performance, such as accuracy or mean squared error.

- What **machine learning model** are you using?

AI Mode All Images Videos Short videos More ▾



param_grid, scoring_func, n_jobs=None, verbose=1, cv=None...



YouTube · DigitalSreeni



302 - Tuning deep learning hyperparameters using ...

for today's uh tutorial. and a bit of a notes before jumping in the grid. search provided by grid search CV. yeah CV stands for cr...

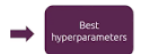


AlliedOffsets



Mastering Hyperparameter Tuning with GridSearchCV in Python

1. Define the model The first step is to define the model that you want to optimize. In scikit-learn, this can be done using the e...



Show all

Random forest

To implement `GridSearchCV` with a **Random Forest Classifier**, you can use the following code template to tune hyperparameters like tree depth and estimator count.

python

```
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV, train_test_split

# 1. Generate sample data
X, y = make_classification(
    n_samples=1000, n_features=20, n_classes=2, random_state=42
)
X_train, X_test, y_train, y_test = train_test_split(
```



AI Mode All Images Videos Short videos More ▾



```
}

# 4. Set up GridSearchCV
grid_search = GridSearchCV(
    estimator=rf, param_grid=param_grid, cv=5, scoring="accuracy", n_j

)

# 5. Fit the model to find the best parameters
grid_search.fit(X_train, y_train)

# 6. View results
print(f"Best Parameters: {grid_search.best_params_}")
print(f"Best CV Score: {grid_search.best_score_:.4f}")

# 7. Use the best model to predict
best_model = grid_search.best_estimator_
accuracy = best_model.score(X_test, y_test)
```

Use code with caution.



Key Random Forest Hyperparameters to Tune

- **n_estimators** : The number of trees in the forest. More trees generally yield better results but slow down computation.
- **max_depth** : The maximum depth of each tree. Limiting this helps prevent overfitting.
- **min_samples_split** : The minimum number of samples required to split an internal node. Higher numbers prevent trees from creating overly specific rules.
- **max_features** : The number of features to consider when looking for the best split (e.g., 'sqrt', 'log2').



AI Mode All Images Videos Short videos More ▾



⌵ ⌵ — ⌵ ⌵



+

🎤